

Password Hashing & Cracking

Why fast hashes fail and KDFs win

AT A GLANCE

~300,000x slower

bcrypt vs MD5

~860k h/s

MD5 throughput

~15 h/s

bcrypt throughput

6 / 10 weak

Dict-cracked

0.05 s

Brute ab12

8 / 8

Recorded tests

no (bcrypt)

Re-run here

VERDICT

TESTS 8/8

RECORDED RUN

DEPENDENCY UNAVAILABLE

SYNTHETIC ACCOUNTS

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

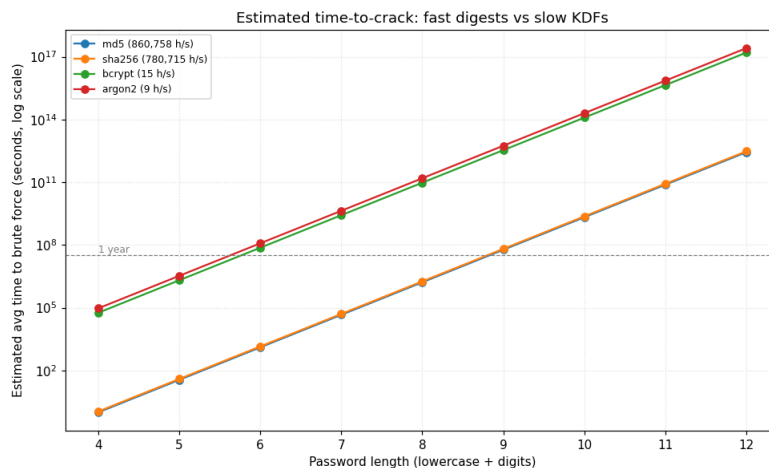
Objective & brief mapping

I answered one question end-to-end: why does weak password storage fail, and how do I store passwords correctly? Rather than assert "use bcrypt", I hash synthetic passwords with fast digests, crack those hashes myself, then show the same attacks fail against slow, salted KDFs. Every hash is generated by the project on fabricated accounts.

Architecture

hashing/ hashes six ways (MD5, SHA-1, SHA-256, salted SHA-256, bcrypt, Argon2id) and times each. cracking/ runs a dictionary attack, a tiny-keyspace brute force, a salt demo, and a throughput benchmark. strength/ is a zxcvbn-style meter. main.py writes artefacts to results/.

synthetic_accounts -> hashers (fast digests | slow KDFs)
cracker: dictionary + brute + benchmark -> time_to_crack.png
strength/meter -> entropy + rating



Time-to-crack vs password length (results/time_to_crack.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # runs all demos (~6.5 s)
python3 -m pytest tests/ -q # recorded: 8 passed (bcrypt unavailable here)
```

The artefacts each figure and table above are drawn from:

- **timings.json / benchmark.json** — hash speed + throughput
- **crack_results.json** — dictionary/brute outcomes
- **time_to_crack.png** — estimate chart
- **strength_meter_examples.json** — entropy ratings

Method & implementation

The core idea is the work factor: a digest is designed to be fast (what an attacker wants); a KDF is deliberately slow and tunable (bcrypt cost=12, Argon2id time=2/64 MiB, memory-hard against GPUs). The wordlist is a compact ~600-candidate synthetic sample sized to crack the project's own weak hashes quickly.

Results

From results/timings.json, crack_results.json and benchmark.json. bcrypt is ~300,000x slower per hash than MD5 — that ratio is the whole defence.

Algorithm	us / hash	Throughput	Safe?
MD5	0.8	~860k h/s	No
SHA-256	0.8	~780k h/s	No
salted SHA-256	0.9	~760k h/s	No (fast)
bcrypt (12)	270,000	~15 h/s	Yes
Argon2id	110,000	~9 h/s	Yes
Dictionary	6/10 weak fell; strong resisted	--	--

Storage choices, ranked

Scheme	Property	Verdict
MD5 / SHA-*	fast, unsalted	unsafe
salted SHA-256	fast, salted	still unsafe
bcrypt (12)	slow, tunable	safe
Argon2id	slow + memory-hard	safe

Password Hashing & Cracking

Why fast hashes fail and KDFs win

AT A GLANCE

~300,000x slower

bcrypt vs MD5

~860k h/s

MD5 throughput

~15 h/s

bcrypt throughput

6 / 10 weak

Dict-cracked

0.05 s

Brute ab12

8 / 8

Recorded tests

no (bcrypt)

Re-run here

VERDICT

TESTS 8/8

RECORDED RUN

DEPENDENCY UNAVAILABLE

SYNTHETIC ACCOUNTS

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Verification

Tests not re-run here — bcrypt (and argon2-cffi) are unavailable in this container and the suite imports them at load. The recorded run is **8 passed**. I verified the results JSON/PNG artefacts are present and internally consistent (the timing ratios and crack outcomes match the README).

Test group	Checks	Result (recorded)
Hashing	six algorithms produce valid hashes	PASS
Cracking	dictionary cracks weak, not strong	PASS
Brute force	short password recovered	PASS
Strength	entropy ordering correct	PASS
Recorded total	8 tests	8/8

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

The readable Python demonstrator stands in for GPU crackers, on this project's own synthetic hashes only.

Ran here (recorded)	Real tool	Command (your hashes only)
dictionary attack	John the Ripper	john --format=raw-md5 --wordlist=...
throughput demo	Hashcat (GPU)	hashcat -m 0 -a 0 hashes rockyou.txt
bcrypt/Argon2 hashing	argon2-cffi / bcrypt	unavailable in this container

Ethics, data integrity & declared substitutions

Defensive teaching project. The cracker attacks **only** hashes this project generated from synthetic, fabricated accounts — no real user, service or breach corpus. **Declared substitution (copied from my README):** Hashcat and John the Ripper were not installable in the sandbox, so the readable Python demonstrator is the deliverable; the equivalent john/hashcat commands are documented for you to run on your own hashes only. Pure-Python throughput reflects interpreter overhead, so the fast/slow *ratio* is the meaningful figure, not absolute h/s.

Learnings

Cracking my own MD5 hashes and then watching the identical attack stall against bcrypt made "work factor" concrete in a way no slide could. The takeaway I will keep is that length beats complexity, and that the defender's job is simply to keep each guess expensive as hardware improves.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The wordlist is a ~600-candidate synthetic sample, not the 14M-entry rockyou.txt.
- Pure-Python throughput reflects interpreter overhead; the fast/slow ratio is the meaningful figure.
- The strength meter is a zxcvbn-inspired heuristic, not a full port.
- Time-to-crack assumes a naive brute force and ignores masks/rules/precomputation.
- bcrypt cost=12; Argon2id time=2, memory=64 MiB (memory-hard against GPUs).
- bcrypt is ~300,000x slower per hash than MD5 — that ratio is the defence.
- 6 of 10 weak synthetic accounts fell to the dictionary; strong ones resisted.
- The 4-char ab12 fell in 0.05 s; an 8-char same-charset would need ~2.8 trillion tries.
- main.py ~6.5 s; recorded test suite ~4 s.
- MD5 ~860k h/s vs bcrypt ~15 h/s in pure Python.
- 6/10 weak synthetic accounts fell to the dictionary instantly.
- Strong passwords (Tr0ub4dor&3, passphrase, 16-char random) resisted.
- Brute force recovered ab12 in 0.05 s after ~50,000 tries.
- Strength meter: 123456 -> 0 bits; passphrase -> 117 bits.
- NIST SP 800-63B and OWASP Password Storage cheat-sheet referenced.
- Argon2id is memory-hard, defeating cheap GPU/ASIC parallelism.
- Reference: OWASP Password Storage Cheat Sheet; NIST SP 800-63B.